

# ITEM 4723 Information Systems Design

## Assignment 2: SQL and the Warehouse Management System

---

### Overview

This assignment exercises is based on the Warehouse Performance Dataset that was used for the inclass examples. As an extension to what was already covered in class, you are required to implement a given relational design in MySQL, populate it with your own personalized copy of the sample data, and then answer a set of representative user questions with SQL queries. Everything in this assignment is SQL. There is no Docker or Git work beyond bringing up the MySQL container you already use for the in-class tutorials.

The database supports the warehouse operations of an online apparel retailer. A short description of those operations is provided in [WarehouseOperations.pdf](#), included alongside this assignment, and you should read it first for context. The nine tables below form the core of the backend database that supports the warehouse management system, or WMS.

### The WMS Relational Design

- **Vendor** (Vendor\_Num, Vendor\_Name, Vendor\_Address, Vendor\_Phone, Vendor\_Email, Vendor\_City, Vendor\_State, Vendor\_ZIP)
- **Shipment** (Shipment\_Num, Vendor\_Num, Shipment\_ReceiptDate, Shipment\_NumPallets, PurchaseOrder\_Num)
- **Cases** (Case\_ID, Item\_Quantity, Case\_Weight)
- **Item** (Item\_ID, Item\_Description, Item\_Price, Item\_Weight, Item\_AnnualSales, Item\_StockLocation)
- **Pallet** (Pallet\_ID, Pallet\_Weight, Pallet\_Volume)
- **Customer** (Customer\_Num, Customer\_FirstName, Customer\_LastName, Customer\_Address, Customer\_City, Customer\_State, Customer\_ZIP, Customer\_Phone, Customer\_Email)
- **CustomerOrder** (Order\_Num, Customer\_Num, Order\_Date, Order\_ReqShipDate, Order\_ActualShipDate, Order\_Carrier)
- **Order\_Item** (Order\_Num, Item\_ID, Item\_Qty)
- **Shipment\_Info** (Shipment\_Num, Case\_ID, Item\_ID, Pallet\_ID, Item\_Qty)

Underlined columns form the primary key of each table. The sample data is provided in the **data/** folder accompanying this assignment, with one CSV file per table.

### Required Before You Begin: Personalize Your Keys

Every student's database must hold slightly different data, so that the query results you submit are your own. Let **xx** be the last two digits of your CWID.

**Every Customer\_Num value and every Item\_ID value in your database must carry the suffix \_xx.** If your CWID ends in 47, then customer C25 is stored as C25\_47, and item PL638-JD is stored as PL638-JD\_47.

Build this in from the very beginning. Do not load the data first and alter it afterwards. Two consequences follow, and you are expected to work both of them out yourself.

- a. **Customer\_Num** and **Item\_ID** are each a primary key in one table and a foreign key in others. The suffix must be applied consistently everywhere those values appear, or your foreign key constraints will reject the data.

- b. The column widths given below are sized for the original values. Widen any column that now has to hold a longer value, at the point where you declare it.

### Part 1: Create the Database and Schema [24 points]

1. Create a database named `WMS_<your CWID>`, substituting your own CWID.
2. Create all nine tables shown above, using exactly the data types listed below. Declare the primary key of every table, and declare every foreign key with `ON UPDATE CASCADE` and `ON DELETE CASCADE`.
  - `CHAR(5)` for `Vendor_Num`, `Vendor_ZIP`, and `Customer_ZIP`
  - `CHAR(10)` for `Vendor_Phone` and `Customer_Phone`
  - `CHAR(2)` for `Vendor_State` and `Customer_State`
  - `CHAR(8)` for `Order_Num`, `CHAR(9)` for `Shipment_Num`, `CHAR(3)` for `Case_ID`
  - `VARCHAR(3)` for `Pallet_ID`, `VARCHAR(5)` for `Customer_Num`
  - `DATE` for `Shipment_ReceiptDate`, `Order_Date`, `Order_ReqShipDate`, and `Order_ActualShipDate`
  - `INTEGER` for `Shipment_NumPallets`, `Item_Quantity`, `Case_Weight`, `Item_Price`, `Item_Weight`, `Item_AnnualSales`, `Pallet_Weight`, `Pallet_Volume`, and `Item_Qty`
  - `VARCHAR(255)` for everything else
3. Decide the order in which you create the tables. A table cannot declare a foreign key that references a table which does not exist yet.

### Part 2: Populate the Tables [20 points]

1. Load the sample data from the `data/` folder into your nine tables, applying your `_xx` suffix to `Customer_Num` and `Item_ID` as you go. Reading each CSV with `csv.DictReader` and appending the suffix in Python before inserting, as in the Lecture 8 tutorial, is the most direct route.
2. Load the tables in an order that respects your foreign keys. A foreign key error during loading usually means you are loading a child table before its parent.
3. Populate at least one table through phpMyAdmin instead of Python, either by writing `INSERT INTO ... VALUES` statements yourself in the SQL tab or by using the Import tab. Include a screenshot showing that table populated in phpMyAdmin.
4. Confirm the row counts in all nine tables, and confirm by inspection that your suffix appears in `Customer`, `CustomerOrder`, `Item`, `Order_Item`, and `Shipment_Info`.

### Part 3: WMS Queries [36 points]

Write one SQL query to answer each request below. Where a request says to use a particular value, pick any value present in your data and state which one you used. Remember that your `Customer_Num` and `Item_ID` values now carry your suffix.

1. List the items (`Item_ID`, `Item_Description`, `Item_Qty`) ordered by a given customer (use `Customer_Num`).
2. List the customers (`Customer_Num`, `Customer_FirstName`, `Customer_LastName`) who ordered a given item (use `Item_ID`).
3. List the customer orders (all attributes) that were shipped late.

4. List the items (Item\_ID, Item\_Description) ordered within a given period, entering a start and end date.
5. List the items (Item\_ID, Item\_Description) with annual sales sorted from highest to lowest.
6. List the items (Item\_ID, Item\_Description) that share a stock location with at least one other item.
7. List the items (Item\_ID, Item\_Description) supplied by a given vendor (use Vendor\_Num).
8. List the vendors (Vendor\_Num, Vendor\_Name) who supplied a given item (use Item\_ID).
9. List the items (Item\_ID, Item\_Description) received within a given period, entering a start and end date.
10. List the pallets (Pallet\_ID) associated with a given vendor (use Vendor\_Num).
11. List the cases (Case\_ID) associated with a given vendor (use Vendor\_Num).
12. List the items (Item\_ID, Item\_Description, Item\_Qty) associated with a given purchase order (use PurchaseOrder\_Num).

#### **Part 4: Your Own Questions [20 points]**

Compose four additional questions of your own that a warehouse manager might reasonably ask of this database, and write the SQL to answer each one. State the question in plain English above each query. At least one of your four must require data from three or more tables.

#### **Deliverables**

Submit the following through Canvas. Do not submit your work to the course GitHub repository.

1. A Jupyter notebook named `assignment2_<yourCUID>.ipynb`, structured like the tutorial notebooks, containing your work for every part in order under a markdown heading per part. Each SQL statement should be held in a clearly named query string and run through your cursor, and the notebook should be saved with its output visible.
2. Screenshots showing the one table you populated through phpMyAdmin in Part 2, and the results of any three queries of your choice from Part 3 re-run in the phpMyAdmin SQL tab.
3. A short note at the top of your notebook giving the two digit suffix you used.

#### **Grading**

This assignment is graded out of 100 points as shown above, which accounts for 60% of your grade for this assignment. The remaining 40% comes from a short, three-question viva, where you will be asked to explain specific choices in your submission, such as why a particular join was needed or why the tables had to be loaded in the order you chose.